

Single-node mode: why the MIP bypass was wrong

OLD: HAND-FIX EVERYTHING

- 1x1 NoC bypassed the MIP solver
- hand-fixed node tile size (node_dims)
- ALSO hand-fixed DRAM order (dram_order)

1

HARDWARE ONLY FIXES CAPACITY

- node has a fixed per-dim capacity - real
- DRAM streaming ORDER is not fixed by hw
 - that's still a search problem

2

REAL MIP CONSTRAINT

- node_dim_capacity (node_capacity.py)
- bounds what fits at NodeLevel only
- solver stays free to search DRAM order

3

NET EFFECT

- single_node = pure topology flag
- no longer hijacks the solve
- schedule is always fully MIP-solved

4

Single-node mode: collapsing the GB hop

NORMAL: TWO-HOP TRAFFIC

- DRAM -> Global Buffer -> node
- reverse for stores: node -> GB -> DRAM
- two physical hops, every time

1

SINGLE-NODE: NO GB EXISTS

- only one PE, zero spatial unrolling
 - `single_node: true` = hardware-topology fact
- not a scheduling choice

2

COLLAPSE TO ONE DIRECT HOP

- DRAM->GB->node becomes DRAM<->node
- same collapse for stores

3

WHAT STAYS UNTOUCHED

- says nothing about how schedule is made
- MIP still freely solves DRAM permutation
- only changes how traffic is ROUTED

4

Implementation: keeping dependencies correct after the collapse

REDIRECT, DON'T REROUTE

- combine() gets an optional arch param
- single_node: src/dest port
-> DRAM directly
- GB-mediated calls skipped
(dram*_id = None)

1

THE ORDERING THAT BROKE

- normally: store-to-DRAM
then load gives order
- single-node: no separate
store-to-DRAM step
- it happened one level up,
in the noc_i loop

2

NEW 4TH DEPENDENCY BRANCH

- if single_node + prior
direct-store exists:
- add it as a dependency
- pulled from ps_hist /
vs_hist instead

3

VERIFIED, NOT ASSUMED

- eventsim: zero unresolved-
dependency errors
- GB actor id never appears
in the trace
 - multi-PE path: byte-
identical, zero regression

4

From dense formula to real spike traces

TODAY: A DENSE FORMULA

- cycle count = product of loop factors
- zero notion of real spike sparsity
- same tile size -> same cycles, always

1

ARCHITECTURES DIFFER

- SpinalFlow, PTB, LoAS, Prosperity,
- Phi, GustavSNN process spikes differently
- dense formula can't capture any of it

2

PLUGGABLE ArchComputeModel

- Protocol: format_input + compute_cycles
- NodeTileSpec: which trace slice a tile covers
- old formula = just the default plugin

3

FIRST PLUGIN: SPINALFLOW

- reconstructs real spike 'spine', time-sorted
- pulled from a real captured LoAS trace
- cycle count = spine length

4

WHERE THIS SITS

- Part 2 of a bigger effort (Part 1: pages 1-3)
- only SpinalFlow built so far
- 5 more archs: future work, same interface

5

Making a risky change safe: process, not just code

7 CHECKPOINTED TASKS

- one task at a time
- each ends: present the diff for review
- no auto-commit

1

SCOPE NARROWED ON PURPOSE

- verify SpinalFlow STANDALONE first
- against hand-specified tiles + real trace
- live wiring into combine(): deferred

2

ZERO REGRESSION, NON-NEGOTIABLE

- every touched task re-runs the pre-change cmd
- requires byte-identical output, every time

3

GROUND-TRUTH CHECK

- reconstruction checked vs independent spike count
- must match exactly, correct chrono order

4

META-LESSON

- narrow scope + checkpoint review + hard gates
- = many small proven changes, not one risky one

5